



华中科技大学
Huazhong University of Science and Technology



华中科技大学
网络空间安全学院
School of Cyber Science and Engineering, HUST

实验四 机器人循线跟踪

华中科技大学 2021 ROS创新实践教学



日期：2021.04



内容：Waffle仿真循线跟踪

0

概述

安装 Gazebo



订阅 图像话题

消息 发给Opencv

获取 路线中心



发布 运动执行命令

Gazebo 提供了在复杂的室内和室外环境中准确有效地模拟机器人群体的能力。有强大的物理引擎，高品质的图形画面，以及方便的编程和图形界面。

教程将在Gazebo的环境下进行，实验任务要求同学能够**结合turtlebot实体**进行控制。

Gazebo通常在安装ROS时已经安装完毕。可运行下列指令检测安装是否成功：

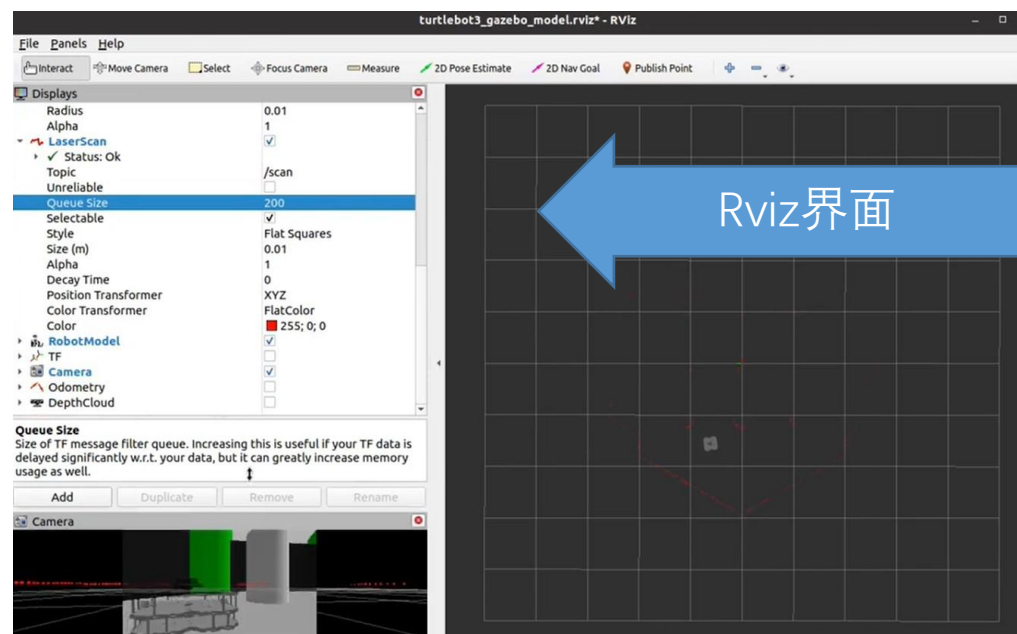
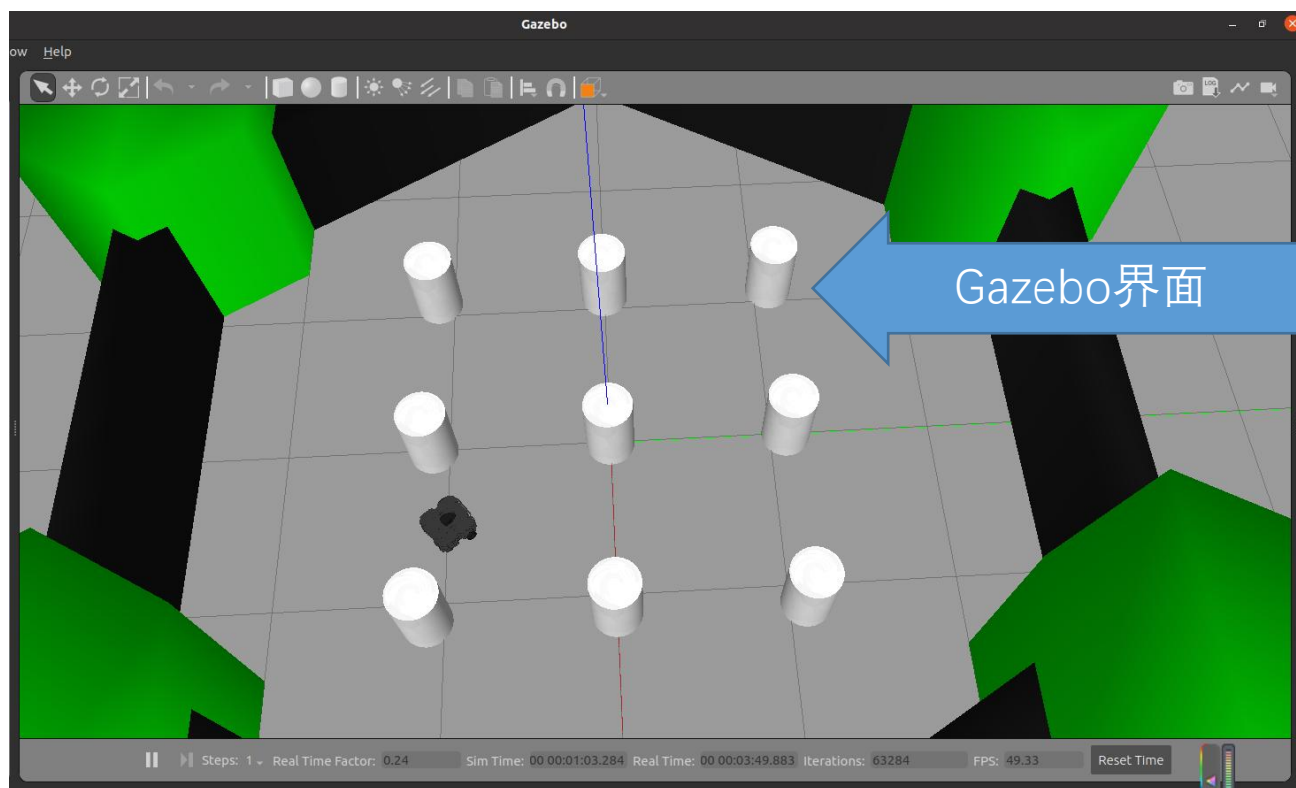
启动turtlebot3_world仿真世界（需要提前下载turtlebot3功能包）

roslaunch turtlebot3_gazebo turtlebot3_world.launch

启动rviz图形化工具

roslaunch turtlebot3_gazebo turtlebot3_gazebo_rviz.launch

启动的正常结果如下图所示。



核心思路：从相机话题提取实时的图像消息。

第一步：查看现有的话题有哪些，找到需要订阅的话题名称。

rostopic list

第二步：生成结点follower，用于订阅话题。

第三步：订阅话题/**camera/rgb/image_raw**。

第四步：编写处理话题消息的机制。

其中二、三、四步，我们通过**python程序**实现。

```
#!/usr/bin/env python3
# 若使用的是python2, 将第一行改成python2
# 若使用的是python, 将第一行改成python
# 之后的程序同理
import rospy
from sensor_msgs.msg import Image # 从传感器图片的库中导入Image数据类型
```

```
def image_callback(msg):
    pass
```

3. 回调函数callback

```
rospy.init_node('follower') # 建立结点follower
image_sub = rospy.Subscriber('camera/rgb/image_raw', Image,
                              image_callback)
```

1. 建立结点follower

2. 订阅话题并设置回调函数

```
# 以上括号内分别是 接受的topic名称, 数据类型, 触发的回调函数
rospy.spin()
```

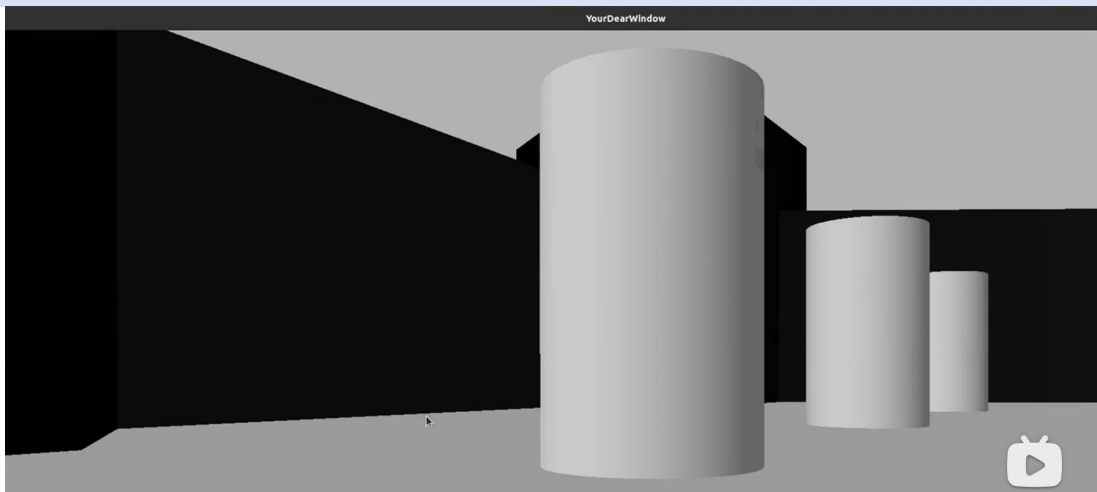
核心思路：通过回调函数接收并处理图像信息，转发给Opencv。

第一步：创建CvBridge对象。

第二步：将消息中的Image数据类型转成OpenCV图像的数据类型。

第三步：用Opencv库的函数显示带有图像的窗口。

期望结果如下。



```
#!/usr/bin/env python3
import rospy
from sensor_msgs.msg import Image
import cv2, cv_bridge # 导入opencv模块和协议模块
```

```
def image_callback(msg):
```

```
    image = bridge.imgmsg_to_cv2(msg,desired_encoding="bgr8")
```

```
    # 将ros_image通过蓝绿红8位色空间转换为OpenCV图像，结果返回给image，类参数bridge转换
```

```
    if(image.all() == None):
```

```
        print("Can't get your image, please check your code!")
```

```
    else :
```

```
        # print(image.size, image.shape) # 输出图像大小以及形状
```

```
        cv2.namedWindow("showImage",cv2.WINDOW_NORMAL) # 建立名为showImage的窗口
```

```
        cv2.imshow("showImage",image[:, :,0]) # 在showImage中显示二维图像
```

```
        cv2.waitKey(3) # waitkey()延时显示图像，在imshow之后，没有waitkey（）函数图像将不会去显示
```

```
rospy.init_node('follower',anonymous = True) # anonymous 同步选项 每公布一条消息就接受一个消息
```

```
bridge = cv_bridge.CvBridge() # 创建CvBridge对象
```

```
image_sub=rospy.Subscriber('/camera/rgb/image_raw',Image,image_callback)
```

```
# 以上括号内分别是 接受的topic名称，数据类型，触发的回调函数
```

```
rospy.spin()
```

2. 将消息中的Image数据类型转成OpenCV图像

3. 用Opencv库的函数显示带有图像的窗口

1. 创建CvBridge对象

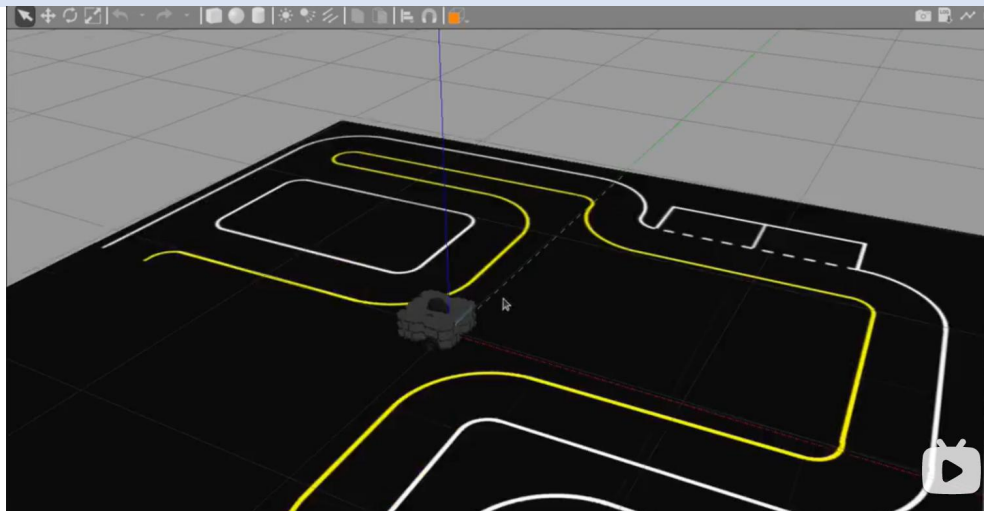
核心思路：用模块model拼接世界，并用launch启动。

第一步：导入需要的模块。

第二步：在world文件中调用模块。

第三步：在launch文件中调用world文件。

期望结果如下。



第一步：教程所用的模块是turtlebot3_simulation库中的course。

找到~/catkin_ws/turtlebot3_simulations/turtlebot3_gazebo/models/，把所有的模块拷贝到自己的功能包目录下。

期望的功能包目录结构如下。

```
├─include
│   └─learning_topic
├─launch
├─models
│   └─turtlebot3_aurorace
│       └─course
│           └─materials
│               ├──scripts
│               └─textures
├─scripts
├─src
└─worlds
```

需要用到的模块

第二步：在world文件中调用模块。

找到~/catkin_ws/turtlebot3_simulations/turtlebot3_gazebo/worlds中的empty.world，复制到自己的worlds文件夹下，并增加对course模块的调用。

```
<!-- Our ground image -->  
<include>  
  <uri>model://turtlebot3_autorace/course</uri>  
  <pose> 0 0 0 0 0 -1.54</pose>  
</include>
```

根据路径调用course模块

第三步：在launch文件中调用world文件。

找到~/catkin_ws/turtlebot3_simulations/turtlebot3_gazebo/launch中的turtlebot3_empty_world.launch，复制到自己的launch文件夹下，并修改world模块的调用路径、增加模块调用路径、增加状态发布结点。

```
<!-- $(find learning_topic)/models/turtlebot3_autorace是你的model文件的路径 -->
<env name="GAZEBO_RESOURCE_PATH" value="$(find learning_topic)/models/turtlebot3_autorace/ground_picture" />
<!-- $(find learning_topic)/worlds/course.world是你的world文件的路径 -->
<arg name="world_name" value="$(find learning_topic)/worlds/course.world"/>

<!-- 增设一个状态发布结点robot_state_publisher -->
<node pkg="robot_state_publisher" type="robot_state_publisher"
name="robot_state_publisher">
  <param name="publish_frequency" type="double" value="30.0" />
</node>
```

添加模块调用的路径

将world路径改成自己的

增加状态发布结点

核心思路：设置颜色上下限，设置蒙版并显示。

第一步：获取图像hsv格式的颜色信息。

第二步：设置hsv格式颜色上下限。

第三步：根据颜色上下限建立蒙版。

期望结果如下。



```
class Follower:
```

```
    def __init__(self):
```

```
        self.bridge = cv_bridge.CvBridge() #创建CvBridge对象
```

```
        self.image_sub=rospy.Subscriber('/camera/rgb/image_raw',Image,self.image_callback)
```

```
        # 函数变量分别为: 接受节点 、 接受消息类型、 回调函数
```

```
    def image_callback(self,msg):
```

```
        image = self.bridge.imgmsg_to_cv2(msg)
```

```
        if(image.all()!=None):
```

```
            hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
```

```
            # 进行消息-> rgb -> hsv格式变量 的两步转换
```

```
            lower_yellow = numpy.array([20,43,46])
```

```
            upper_yellow = numpy.array([90,255,255])
```

```
            # 建立蒙版参量 参量使用指针格式 (inRange函数的要求)
```

```
            mask = cv2.inRange(hsv, lower_yellow, upper_yellow)
```

```
            masked = cv2.bitwise_and(image,image,mask=mask)
```

```
            # 使用蒙版进行二值化 bitwise
```

```
            cv2.namedWindow("showYellowOnly",cv2.WINDOW_NORMAL)
```

```
            cv2.imshow("showYellowOnly",mask) #进行显示
```

```
            cv2.waitKey(3)
```

1. 获取图像hsv格式的颜色信息

2. 设置hsv格式颜色上下限

3. 根据颜色上下限建立蒙版

4. 只显示蒙版部分

```
rospy.init_node('follower',anonymous = True)
```

```
follower = Follower()
```

```
rospy.spin()
```

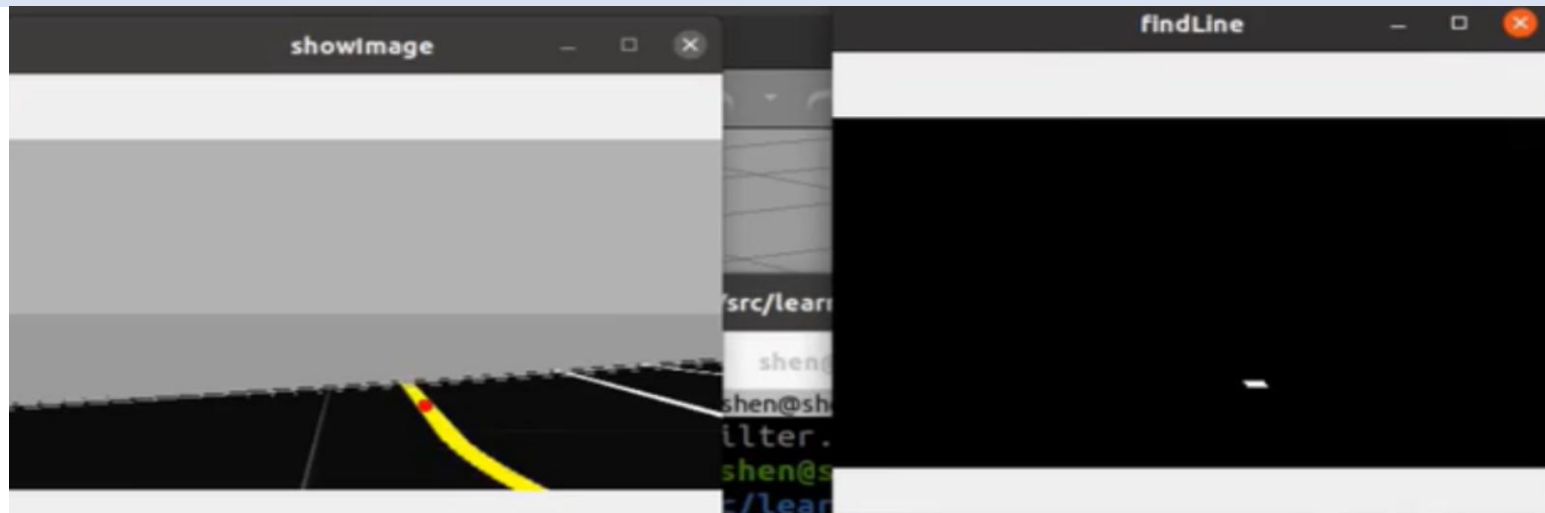
核心思路：根据蒙版对图像进行切片，并用Opencv的moments函数计算路线中心。

第一步：对颜色蒙版切片。

第二步：计算路线中心并标记圆圈。

第三步：显示原图以及路线中心两窗口。

期望结果如下。



```
h,w,d= image.shape
```

```
top = 3*h/4
```

```
bot = top + 20
```

```
# 在图像的下3/4处进行切片 注意: image纵向向下为x正向 横向向右为y正向
```

```
mask[int(0):int(top),:] = 0
```

```
mask[int(bot):int(h),:] = 0
```

```
M = cv2.moments(mask)
```

```
#moments(): Opencv中用于计算图像中心的函数类 参见Opencv官网
```

```
if M['m00']>0:
```

```
    cx = int(M['m10']/M['m00'])
```

```
    cy = int(M['m01']/M['m00'])
```

```
    cv2.circle(image,(cx,cy),20,(0,0,255),-1)
```

```
# 对切片之后的图像计算中心, 并标记小圆圈
```

```
cv2.namedWindow("showImage",cv2.WINDOW_NORMAL)
```

```
cv2.imshow("showImage",image)
```

```
cv2.namedWindow("findLine",cv2.WINDOW_NORMAL)
```

```
cv2.imshow("findLine",mask)
```

```
cv2.waitKey(3)
```

1. 获取基础的常数

2. 对颜色蒙版切片

3. 计算路线中心并标记圆圈

4. 显示原图以及路线中心两窗口

核心思路：基于路线中心，向仿真的小车发布消息使其向着路线中心运动。

第一步：获取运动相关的话题。

第二步：根据路线中心，发布运动消息。

期望结果：小车成功地沿着黄线运动。



```
def __init__(self):  
    self.bridge = cv_bridge.CvBridge()  
    self.image_sub = rospy.Subscriber('/camera/rgb/image_raw', Image, self.image_callback)  
    self.cmd_vel_pub = rospy.Publisher('cmd_vel', Twist, queue_size=1) # 发布话题使小车运动  
    self.twist = Twist()
```

1. 获取话题，准备向它Publish消息

```
if M['m00'] > 0:  
    cx = int(M['m10']/M['m00'])  
    cy = int(M['m01']/M['m00'])  
    cv2.circle(image, (cx, cy), 20, (0, 0, 255), -1)  
    err = cx - w/2  
    self.twist.linear.x = 0.15 # 小车的线速度不易设置太快，否则容易脱离跑道  
    self.twist.angular.z = -float(err)/1000 # 小车运动的角度  
    self.cmd_vel_pub.publish(self.twist)  
cv2.namedWindow("showImage", cv2.WINDOW_NORMAL)  
cv2.imshow("showImage", image)  
cv2.waitKey(3)
```

2. 根据路线中心，发布运动消息

The background features a large, light blue downward-pointing triangle. Inside it is a smaller, thin-lined upward-pointing triangle. A thick blue arrow points from the upper left towards the center, and another thick blue arrow points downwards from the center. The text "THANK YOU" is centered in a bold, blue, sans-serif font.

THANK YOU